

Egendefinerte typer (kap. 8)

Egendefinerte typer (kap. 8)

- ▶ Synonymer: `type Intpair = (Int,Int)`
(kan brukes om hverandre)

Egendefinerte typer (kap. 8)

- ▶ Synonymer: `type` `Intpair = (Int,Int)`
(kan brukes om hverandre)
- ▶ Disjunkte: `newtype` `Nat = N Int` (1 konstruktør/1 argument!)

Egendefinerte typer (kap. 8)

- ▶ Synonymer: `type Intpair = (Int,Int)`
(kan brukes om hverandre)
- ▶ Disjunkte: `newtype Nat = N Int` (1 konstruktør/1 argument!)
- ▶ Nye typer: `data Direction = Ne | Se | Sw | Nw`

Egendefinerte typer (kap. 8)

- ▶ Synonymer: `type Intpair = (Int,Int)`
(kan brukes om hverandre)
- ▶ Disjunkte: `newtype Nat = N Int` (1 konstruktør/1 argument!)
- ▶ Nye typer: `data Direction = Ne | Se | Sw | Nw`

```
data BTreeI = Leaf Int | Node BTreeI BTreeI
```

Egendefinerte typer (kap. 8)

- ▶ Synonymer: `type Intpair = (Int,Int)`
(kan brukes om hverandre)
- ▶ Disjunkte: `newtype Nat = N Int` (1 konstruktør/1 argument!)
- ▶ Nye typer: `data Direction = Ne | Se | Sw | Nw`

```
data BTreeI = Leaf Int | Node BTreeI BTreeI
```

(entydige konstruktornavn: ... | Node Int – feil)

Egendefinerte typer (kap. 8)

- ▶ Synonymer: `type Intpair = (Int,Int)`
(kan brukes om hverandre)
- ▶ Disjunkte: `newtype Nat = N Int` (1 konstruktør/1 argument!)
- ▶ Nye typer: `data Direction = Ne | Se | Sw | Nw`

```
data BTreI = Leaf Int | Node BTreI BTreI
```

(entydige konstruktørnavn: ... | Node Int – feil)

- ▶ `data` muliggjør flere egne konstruktører.
Navn av typer og konstruktører må starte med storbokstav!

Egendefinerte typer (kap. 8)

- ▶ Synonymer: `type Intpair = (Int,Int)`
(kan brukes om hverandre)
- ▶ Disjunkte: `newtype Nat = N Int` (1 konstruktør/1 argument!)
- ▶ Nye typer: `data Direction = Ne | Se | Sw | Nw`

`data BTreI = Leaf Int | Node BTreI BTreI`
(entydige konstruktørnavn: ... | Node Int – feil)

- ▶ `data` muliggjør flere egne konstruktører.
Navn av typer og konstruktører må starte med storbokstav!

Egendefinerte typer (kap. 8)

- ▶ Synonymer: `type Intpair = (Int,Int)`
(kan brukes om hverandre)
- ▶ Disjunkte: `newtype Nat = N Int` (1 konstruktør/1 argument!)
- ▶ Nye typer: `data Direction = Ne | Se | Sw | Nw`
typen blir en instans av en klasse ved å si: **deriving** (Eq, Ord)

```
data BTreeI = Leaf Int | Node BTreeI BTreeI
```

(entydige konstruktornavn: ... | Node Int – feil)

- ▶ `data` muliggjør flere egne konstruktører.
Navn av typer og konstruktører må starte med storbokstav!

Egendefinerte typer (kap. 8)

- ▶ Synonymer: `type Intpair = (Int,Int)`
(kan brukes om hverandre)
- ▶ Disjunkte: `newtype Nat = N Int` (1 konstruktør/1 argument!)
- ▶ Nye typer: `data Direction = Ne | Se | Sw | Nw`
typen blir en instans av en klasse ved å si: **deriving** (Eq, Ord)

```
data BTreeI = Leaf Int | Node BTreeI BTreeI
```

(entydige konstruktornavn: ... | Node Int – feil)

- ▶ `data` muliggjør flere egne konstruktører.
Navn av typer og konstruktører må starte med storbokstav!
- ▶ For enkle typer, funksjoner som arves ved **deriving**
– fra Show, Read, Eq, Ord, Enum, Bounded –
blir automatisk avledet av Haskell. Man kan dog omdefinere dem, dersom ønskelig.

Typer er (oftest) instanser av typeklasser (kap.8.5)

- **Show**, med en operasjon `show :: a -> String` (viser ikke!)
- Read**, med en operasjon `read :: String -> a` (leser ikke!)

Typer er (oftest) instanser av typeklasser (kap.8.5)

- ▶ **Show**, med en operasjon `show :: a -> String` (viser ikke!)
- ▶ **Read**, med en operasjon `read :: String -> a` (leser ikke!)
- ▶ **Eq**, med `==,/= :: a -> a -> Bool`

Typer er (oftest) instanser av typeklasser (kap.8.5)

- ▶ **Show**, med en operasjon `show :: a -> String` (viser ikke!)
 - ▶ **Read**, med en operasjon `read :: String -> a` (leser ikke!)
 - ▶ **Eq**, med `==, /= :: a -> a -> Bool`
 - ▶ **Num**, `(+)`, `(-)`, `(*)`, `abs`, `negate...`,
 - en subklasse av `Eq` og `Show` med subklasser som
 - ▶ **Integral**, **Fractional**, ...
- > :t 1
Num a => a

Typer er (oftest) instanser av typeklasser (kap.8.5)

- ▶ **Show**, med en operasjon `show :: a -> String` (viser ikke!)
- ▶ **Read**, med en operasjon `read :: String -> a` (leser ikke!)
- ▶ **Eq**, med `==, /= :: a -> a -> Bool`
- ▶ **Num**, `(+)`, `(-)`, `(*)`, `abs`, `negate...`,
 - en subklasse av `Eq` og `Show` med subklasser som
 - ▶ **Integral**, **Fractional**, ...
 - > `:t 1`
`Num a => a`
- ▶ **Ord**, en subklasse av `Eq`, med instanser (typer) som er totalt ordnet; operasjoner som
 - `(<)`, `(<=)`, `(>)`, `(>=)` `:: a -> a -> Bool`
 - `min, max :: a -> a -> a`

Typer er (oftest) instanser av typeklasser (kap.8.5)

- ▶ **Show**, med en operasjon `show :: a -> String` (viser ikke!)
- Read**, med en operasjon `read :: String -> a` (leser ikke!)
- ▶ **Eq**, med `==, /= :: a -> a -> Bool`
- ▶ **Num**, `(+)`, `(-)`, `(*)`, `abs`, `negate...`,
 - en subklasse av `Eq` og `Show` med subklasser som
 - ▶ **Integral**, **Fractional**, ...
 - > `:t 1`
`Num a => a`
- ▶ **Ord**, en subklasse av `Eq`, med instanser (typer) som er totalt ordnet; operasjoner som
 - `(<)`, `(<=)`, `(>)`, `(>=)` `:: a -> a -> Bool`
 - `min, max :: a -> a -> a`
 - > `:t (<)`

Typer er (oftest) instanser av typeklasser (kap.8.5)

- ▶ **Show**, med en operasjon `show :: a -> String` (viser ikke!)
- Read**, med en operasjon `read :: String -> a` (leser ikke!)
- ▶ **Eq**, med `==, /= :: a -> a -> Bool`
- ▶ **Num**, `(+)`, `(-)`, `(*)`, `abs`, `negate...`,
 - en subklasse av `Eq` og `Show` med subklasser som
 - ▶ **Integral**, **Fractional**, ...
 - > `:t 1`
`Num a => a`
- ▶ **Ord**, en subklasse av `Eq`, med instanser (typer) som er totalt ordnet; operasjoner som
 - `(<)`, `(<=)`, `(>)`, `(>=)` `:: a -> a -> Bool`
 - `min, max :: a -> a -> a`
 - > `:t (<)`
`Ord a => a -> a -> Bool`

Typer er (oftest) instanser av typeklasser (kap.8.5)

- ▶ **Show**, med en operasjon `show :: a -> String` (viser ikke!)
- Read**, med en operasjon `read :: String -> a` (leser ikke!)
- ▶ **Eq**, med `==, /= :: a -> a -> Bool`
- ▶ **Num**, `(+)`, `(-)`, `(*)`, `abs`, `negate...`,
 - en subklasse av `Eq` og `Show` med subklasser som
 - ▶ **Integral**, **Fractional**, ...
 - > `:t 1`
`Num a => a`
- ▶ **Ord**, en subklasse av `Eq`, med instanser (typer) som er totalt ordnet; operasjoner som
 - `(<)`, `(<=)`, `(>)`, `(>=)` `:: a -> a -> Bool`
 - `min, max :: a -> a -> a`
 - > `:t (<)`
`Ord a => a -> a -> Bool`
- ▶ ...

Typer er (oftest) instanser av typeklasser (kap.8.5)

- ▶ **Show**, med en operasjon `show :: a -> String` (viser ikke!)
- Read**, med en operasjon `read :: String -> a` (leser ikke!)
- ▶ **Eq**, med `==, /= :: a -> a -> Bool`
- ▶ **Num**, `(+)`, `(-)`, `(*)`, `abs`, `negate...`,
 - en subklasse av `Eq` og `Show` med subklasser som
 - ▶ **Integral**, **Fractional**, ...
 - > `:t 1`
`Num a => a`
- ▶ **Ord**, en subklasse av `Eq`, med instanser (typer) som er totalt ordnet; operasjoner som
 - `(<)`, `(<=)`, `(>)`, `(>=)` `:: a -> a -> Bool`
 - `min, max :: a -> a -> a`
 - > `:t (<)`
`Ord a => a -> a -> Bool`
- ▶ ...
- ▶ Alle basistyper er instanser av **Eq**, **Ord**, **Show** og **Read**.
Også lister og tupler er slike typer, hvis bare deres element-/komponent-typer er det.

Typer er (oftest) instanser av typeklasser

► `class Eq t where`
 `(==), (/=) :: t -> t -> Bool`
 `x /= y = not(x == y)`

Typer er (oftest) instanser av typeklasser

- ▶ `class Eq t where`
 `(==), (/=) :: t -> t -> Bool`
 `x /= y = not(x == y)`
- ▶ klasser kan ha subklasser
 `class Eq t => Ord t where`
 `(>), (<), (>=), (<=) :: t -> t -> Bool`
 `x > y = y < x`
 `x <= y = (x < y) || (x == y)`
 ...

Typer er (oftest) instanser av typeklasser

► **class** Eq t **where**

(==), (/=) :: t -> t -> Bool

x /= y = not(x == y)

► klasser kan ha subklasser

class Eq t => Ord t **where**

(>), (<), (>=), (<=) :: t -> t -> Bool

x > y = y < x

x <= y = (x < y) || (x == y)

...

► typer deklart med **data** kan bli instanser av klasser

(a) **instance** Ord Bool **where**

False < True = True

(b) **data** Bool = False | True **deriving** (Eq, Ord)

Typer er (oftest) instanser av typeklasser

► `class Eq t where`

`(==), (/=) :: t -> t -> Bool`

`x /= y = not(x == y)`

► klasser kan ha subklasser

`class Eq t => Ord t where`

`(>), (<), (>=), (<=) :: t -> t -> Bool`

`x > y = y < x`

`x <= y = (x < y) || (x == y)`

...

► typer deklart med `data` kan bli instanser av klasser

(a) `instance Ord Bool where`

`False < True = True`

(b) `data Bool = False | True deriving (Eq, Ord)`

... og dette blir nok tilstrekkelig for vår del...

Våre typer blir (vanligvis) instanser av Show

► data Dir = Ne | Nw | Se | Sw deriving Show

Våre typer blir (vanligvis) instanser av Show

- ▶ `data Dir = Ne | Nw | Se | Sw deriving Show`
- > `show Ne`
 `"Ne"`

Våre typer blir (vanligvis) instanser av Show

- ▶ `data Dir = Ne | Nw | Se | Sw deriving Show`
- > `show Ne`
 `"Ne"`
- ▶ `data Dir = Ne | Nw | Se | Sw`
 `instance Show Dir where`
 `show Ne = "north east"`
 `show Nw = "north west"`

Våre typer blir (vanligvis) instanser av Show

- ▶ data Dir = Ne | Nw | Se | Sw deriving Show
- > show Ne
"Ne"
- ▶ data Dir = Ne | Nw | Se | Sw
instance Show Dir where
 show Ne = "north east"
 show Nw = "north west"
- > show Ne
"north east"

Våre typer blir (vanligvis) instanser av Show

- ▶ data Dir = Ne | Nw | Se | Sw deriving Show
- > show Ne
"Ne"
- ▶ data Dir = Ne | Nw | Se | Sw
instance Show Dir where
 show Ne = "north east"
 show Nw = "north west"
- > show Ne
"north east"
- > show Se
Exception: ... Non-exhaustive pattern in function show

Våre typer blir (vanligvis) instanser av Show

- ▶ `data Dir = Ne | Nw | Se | Sw deriving Show`
- > `show Ne`
 `"Ne"`
- ▶ `data Dir = Ne | Nw | Se | Sw`
 `instance Show Dir where`
 `show Ne = "north east"`
 `show Nw = "north west"`
- > `show Ne`
 `"north east"`
- > `show Se`
 Exception: ... Non-exhaustive pattern in function show
- ▶ `myshow Se = "south east"`

Våre typer blir (vanligvis) instanser av Show

- ▶ `data Dir = Ne | Nw | Se | Sw deriving Show`
- > `show Ne`
 `"Ne"`
- ▶ `data Dir = Ne | Nw | Se | Sw`
 `instance Show Dir where`
 `show Ne = "north east"`
 `show Nw = "north west"`
- > `show Ne`
 `"north east"`
- > `show Se`
 Exception: ... Non-exhaustive pattern in function show
- ▶ `myshow Se = "south east"`
 `myshow x = show x`

Egendefinerte forkortelser

► `type` Intpair = (Int,Int)

Egendefinerte forkortelser

- ▶ `type` Intpair = (Int,Int)
- ▶ `newtype` Nonneg = N Int

Egendefinerte forkortelser

► `type` Intpair = (Int,Int)

► `newtype` Nonneg = N Int

► Parametriserte typer:

For gitte typer `Nok` og `Verdi`, kan vi deklareere en ny:

`type` Ordbok = [(Nok, Verdi)]

Egendefinerte forkortelser

► `type` Intpair = (Int,Int)

► `newtype` Nonneg = N Int

► Parametriserte typer:

For gitte typer Nok og Verdi, kan vi deklarere en ny:

`type` Ordbok = [(Nok, Verdi)]

Egendefinerte forkortelser

► `type` Intpair = (Int,Int)

► `newtype` Nonneg = N Int

► Parametriserte typer:

For gitte typer Nok og Verdi, kan vi deklarere en ny:

```
type Ordbok = [ (Nok, Verdi) ]
```

helst `Eq Nok`, siden den skal brukes ved søking

```
find k obk =
```

Egendefinerte forkortelser

► **type** Intpair = (Int,Int)

► **newtype** Nonneg = N Int

► Parametriserte typer:

For gitte typer Nok og Verdi, kan vi deklareere en ny:

type Ordbok = [(Nok, Verdi)]

helst **Eq** **Nok**, siden den skal brukes ved søking

find k obk = [v | (n,v) <- obk, **n == k**]

Egendefinerte forkortelser

► **type** Intpair = (Int,Int)

► **newtype** Nonneg = N Int

► Parametriserte typer:

For gitte typer Nok og Verdi, kan vi deklare en ny:

type Ordbok = [(Nok, Verdi)]

helst **Eq Nok**, siden den skal brukes ved søking

find k obk = **head** [v | (n,v) <- obk, n == k]

Egendefinerte forkortelser

► **type** Intpair = (Int,Int)

► **newtype** Nonneg = N Int

► **Parametriserte** typer:

For gitte typer Nok og Verdi, kan vi deklare en ny:

type Ordbok = [(Nok, Verdi)]

helst Eq Nok, siden den skal brukes ved søking

find k obk = head [v | (n,v) <- obk, n == k]

– parametriser typen (med typevariabler)

Egendefinerte forkortelser

▶ **type** Intpair = (Int,Int)

▶ **newtype** Nonneg = N Int

▶ **Parametriserte** typer:

For gitte typer Nok og Verdi, kan vi deklare en ny:

type Ordbok = [(Nok, Verdi)]

helst Eq Nok, siden den skal brukes ved søking

find k obk = head [v | (n,v) <- obk, n == k]

– parametriser typen (med typevariabler)

type Ordbok n v = [(n, v)]

Egendefinerte forkortelser

► **type** Intpair = (Int,Int)

► **newtype** Nonneg = N Int

► Parametriserte typer:

For gitte typer Nok og Verdi, kan vi deklare en ny:

type Ordbok = [(Nok, Verdi)]

helst Eq Nok, siden den skal brukes ved søking

find k obk = head [v | (n,v) <- obk, n == k]

– parametriser typen (med typevariabler)

type Ordbok n v = [(n, v)]

► **data** Tre = Leaf Int | Node Tre Int Tre

– egne konstruktorer av datatypen krever nøkkelordet **data...**

Egendefinerte forkortelser

► `type Intpair = (Int,Int)`

► `newtype Nonneg = N Int`

► Parametriserte typer:

For gitte typer `Nok` og `Verdi`, kan vi deklareere en ny:

`type Ordbok = [ (Nok, Verdi) ]`

helst `Eq Nok`, siden den skal brukes ved søking

`find k obk = head [ v | (n,v) <- obk, n == k ]`

– parametriser typen (med typevariabler)

`type Ordbok n v = [ (n, v) ]`

► `data Tre = Leaf Int | Node Tre Int Tre`

– egne konstruktorer av datatypen krever nøkkelordet `data...`

Egendefinerte forkortelser

► `type` Intpair = (Int,Int)

► `newtype` Nonneg = N Int

► Parametriserte typer:

For gitte typer Nok og Verdi, kan vi deklareere en ny:

`type` Ordbok = [(Nok, Verdi)]

helst Eq Nok, siden den skal brukes ved søking

`find k obk = head [ v | (n,v) <- obk, n == k ]`

– parametriser typen (med typevariabler)

`type` Ordbok `n v` = [(n, v)]

► `data` Tre = Leaf Int | Node Tre Int Tre

– egne konstruktorer av datatypen krever nøkkelordet `data...`

► `data` Tre `t` = Leaf `t` | Node (Tre `t`) `t` (Tre `t`)

Egendefinerte nye typer: **data**

- ▶ `data Fig = Circ Float | Rek Float Float`

Egendefinerte nye typer: data

- ▶ data `Fig = Circ Float | Rek Float Float`
- ▶ datakonstruktorer er **funksjoner** av **typen** , dvs.

Egendefinerte nye typer: **data**

- ▶ `data Fig = Circ Float | Rek Float Float`
- ▶ datakonstruktorer er **funksjoner** av **typen** , dvs.

Circ :: Float -> Fig

Rek :: Float -> Float -> Fig

Egendefinerte nye typer: **data**

- ▶ `data Fig = Circ Float | Rek Float Float`
- ▶ datakonstruktorer er **funksjoner** av **typen** , dvs.
`Circ :: Float -> Fig`
`Rek :: Float -> Float -> Fig`
- > men de har ikke noen definerende ligninger, fordi
`Rek 2 3` er allerede en ferdigevaluert verdi = `Rek 2 3 :: Fig`

Egendefinerte nye typer: **data**

- ▶ `data Fig = Circ Float | Rek Float Float`
- ▶ datakonstruktorer er funksjoner av typen , dvs.
`Circ :: Float -> Fig`
`Rek :: Float -> Float -> Fig`
- > men de har ikke noen definerende ligninger, fordi
`Rek 2 3` er allerede en ferdigevaluert verdi = `Rek 2 3 :: Fig`
- ▶ Funksjoner kan nå defineres ved bruk av konstruktorer som mønstre:

Egendefinerte nye typer: data

- ▶ `data Fig = Circ Float | Rek Float Float`
- ▶ datakonstruktorer er funksjoner av typen , dvs.
`Circ :: Float -> Fig`
`Rek :: Float -> Float -> Fig`
- > men de har ikke noen definerende ligninger, fordi
`Rek 2 3` er allerede en ferdigevaluert verdi = `Rek 2 3 :: Fig`
- ▶ Funksjoner kan nå defineres ved bruk av konstruktorer som mønstre:
`kvad n = Rek n n`

Egendefinerte nye typer: data

- ▶ data Fig = Circ Float | Rek Float Float
- ▶ datakonstruktorer er funksjoner av typen , dvs.
Circ :: Float -> Fig
Rek :: Float -> Float -> Fig
- > men de har ikke noen definerende ligninger, fordi
Rek 2 3 er allerede en ferdigevaluert verdi = Rek 2 3 :: Fig
- ▶ Funksjoner kan nå defineres ved bruk av konstruktorer som mønstre:
kvad n = Rek n n
area (Rek n m) = n*m

Egendefinerte nye typer: data

- ▶ data Fig = Circ Float | Rek Float Float
- ▶ datakonstruktorer er funksjoner av typen , dvs.
Circ :: Float -> Fig
Rek :: Float -> Float -> Fig
- > men de har ikke noen definerende ligninger, fordi
Rek 2 3 er allerede en ferdigevaluert verdi = Rek 2 3 :: Fig
- ▶ Funksjoner kan nå defineres ved bruk av konstruktorer som mønstre:
kvad n = Rek n n
area (Rek n m) = n*m
area (Circ n) = pi * n^ 2

Egendefinerte nye typer: data

- ▶ `data Fig = Circ Float | Rek Float Float`
- ▶ datakonstruktorer er funksjoner av typen , dvs.
`Circ :: Float -> Fig`
`Rek :: Float -> Float -> Fig`
- > men de har ikke noen definerende ligninger, fordi
`Rek 2 3` er allerede en ferdigevaluert verdi = `Rek 2 3 :: Fig`
- ▶ Funksjoner kan nå defineres ved bruk av konstruktorer som mønstre:
`kvad n = Rek n n`
`area (Rek n m) = n*m`
`area (Circ n) = pi * n^2`
- ▶ `Circ/Rek` kan uformelt oppfates som “subtyper” av `Fig`

Egendefinerte nye typer: data

- ▶ `data Fig = Circ Float | Rek Float Float`
- ▶ datakonstruktorer er funksjoner av typen , dvs.
`Circ :: Float -> Fig`
`Rek :: Float -> Float -> Fig`
- > men de har ikke noen definerende ligninger, fordi
`Rek 2 3` er allerede en ferdigevaluert verdi = `Rek 2 3 :: Fig`
- ▶ Funksjoner kan nå defineres ved bruk av konstruktorer som mønstre:
`kvad n = Rek n n`
`area (Rek n m) = n*m`
`area (Circ n) = pi * n^ 2`
- ▶ `Circ/Rek` kan uformelt oppfates som “subtyper” av `Fig`
- ▶ `data Circ = Ci Int`
`data Fig = Circ | ...`
er ikke lovlig.

Egendefinerte nye typer: **data**

- ▶ `data Fig = Circ Float | Rek Float Float`
- ▶ datakonstruktorer er funksjoner av typen , dvs.
`Circ :: Float -> Fig`
`Rek :: Float -> Float -> Fig`
- > men de har ikke noen definerende ligninger, fordi
`Rek 2 3` er allerede en ferdigevaluert verdi = `Rek 2 3 :: Fig`
- ▶ Funksjoner kan nå defineres ved bruk av konstruktorer som mønstre:
`kvad n = Rek n n`
`area (Rek n m) = n*m`
`area (Circ n) = pi * n^ 2`
- ▶ `Circ/Rek` kan uformelt oppfates som “subtyper” av `Fig`
- ▶ `data Circ = Ci Int`
`data Fig = Konstr Circ | ...`
er lovlig.

Egendefinerte nye typer: data

► `data Ret = Op | Ne | Ve | Ho`

Egendefinerte nye typer: data

▶ `data Ret = Op | Ne | Ve | Ho`

Egendefinerte nye typer: data

- ▶ `data Ret = Op | Ne | Ve | Ho deriving (Eq, Ord, Read, Show)`
- ▶ `type Pos = (Int, Int)`

Egendefinerte nye typer: data

- ▶ `data Ret = Op | Ne | Ve | Ho deriving (Eq, Ord, Read, Show)`
- ▶ `type Pos = (Int, Int)`
- ▶ `mv :: Pos -> Ret -> Pos`
`mv (x,y) Ve = (x-1,y)   mv (x,y) Ho = (x+1,y)`
`mv (x,y) Op = (x,y+1)   mv (x,y) Ne = (x,y-1)`

Egendefinerte nye typer: data

- ▶ $\text{data Ret} = \text{Op} \mid \text{Ne} \mid \text{Ve} \mid \text{Ho}$ deriving (Eq, Ord, Read, Show)
- ▶ $\text{type Pos} = (\text{Int}, \text{Int})$
- ▶ $\text{mv} :: \text{Pos} \rightarrow \text{Ret} \rightarrow \text{Pos}$
 $\text{mv } (x,y) \text{ Ve} = (x-1,y)$ $\text{mv } (x,y) \text{ Ho} = (x+1,y)$
 $\text{mv } (x,y) \text{ Op} = (x,y+1)$ $\text{mv } (x,y) \text{ Ne} = (x,y-1)$
- ▶ $\text{mvs} :: \text{Pos} \rightarrow [\text{Ret}] \rightarrow \text{Pos}$
- - $\text{mvs } p [a,b,c\dots] = \text{fra } p$: først a, deretter b (fra $\text{mv } p a$), så c
 $\text{mvs } p [] = p$

Egendefinerte nye typer: data

- ▶ $\text{data Ret} = \text{Op} \mid \text{Ne} \mid \text{Ve} \mid \text{Ho}$ deriving (Eq, Ord, Read, Show)
- ▶ $\text{type Pos} = (\text{Int}, \text{Int})$
- ▶ $\text{mv} :: \text{Pos} \rightarrow \text{Ret} \rightarrow \text{Pos}$
 $\text{mv } (x,y) \text{ Ve} = (x-1,y)$ $\text{mv } (x,y) \text{ Ho} = (x+1,y)$
 $\text{mv } (x,y) \text{ Op} = (x,y+1)$ $\text{mv } (x,y) \text{ Ne} = (x,y-1)$
- ▶ $\text{mvs} :: \text{Pos} \rightarrow [\text{Ret}] \rightarrow \text{Pos}$
- - $\text{mvs } p [a,b,c\dots] = \text{fra } p$: først a, deretter b (fra $\text{mv } p a$), så c
 $\text{mvs } p [] = p$
 $\text{mvs } p (r:rs) =$

Egendefinerte nye typer: data

- ▶ $\text{data Ret} = \text{Op} \mid \text{Ne} \mid \text{Ve} \mid \text{Ho}$ deriving (Eq, Ord, Read, Show)
- ▶ $\text{type Pos} = (\text{Int}, \text{Int})$
- ▶ $\text{mv} :: \text{Pos} \rightarrow \text{Ret} \rightarrow \text{Pos}$
 $\text{mv } (x,y) \text{ Ve} = (x-1,y)$ $\text{mv } (x,y) \text{ Ho} = (x+1,y)$
 $\text{mv } (x,y) \text{ Op} = (x,y+1)$ $\text{mv } (x,y) \text{ Ne} = (x,y-1)$
- ▶ $\text{mvs} :: \text{Pos} \rightarrow [\text{Ret}] \rightarrow \text{Pos}$
- - $\text{mvs } p [a,b,c\dots] = \text{fra } p$: først a, deretter b (fra $\text{mv } p a$), så c
 $\text{mvs } p [] = p$
 $\text{mvs } p (r:rs) = \text{mvs } (\text{mv } p r) rs$

Egendefinerte nye typer: data

▶ data Ret = Op | Ne | Ve | Ho deriving (Eq, Ord, Read, Show)

▶ type Pos = (Int, Int)

▶ mv :: Pos -> Ret -> Pos

mv (x,y) Ve = (x-1,y) mv (x,y) Ho = (x+1,y)

mv (x,y) Op = (x,y+1) mv (x,y) Ne = (x,y-1)

▶ mvs :: Pos -> [Ret] -> Pos

- - mvs p [a,b,c...] = fra p: først a, deretter b (fra mv p a), så c

mvs p [] = p

mvs p (r:rs) = mvs (mv p r) rs

... = (mv (mv (mv p a) b) c)...

Egendefinerte nye typer: data

▶ data Ret = Op | Ne | Ve | Ho deriving (Eq, Ord, Read, Show)

▶ type Pos = (Int, Int)

▶ mv :: Pos -> Ret -> Pos

mv (x,y) Ve = (x-1,y) mv (x,y) Ho = (x+1,y)

mv (x,y) Op = (x,y+1) mv (x,y) Ne = (x,y-1)

▶ mvs :: Pos -> [Ret] -> Pos

- - mvs p [a,b,c...] = fra p: først a, deretter b (fra mv p a), så c

mvs p [] = p

mvs p (r:rs) = mvs (mv p r) rs

... = (mv (mv (mv p a) b) c)... = foldl mv p [a,b,c...]

Egendefinerte nye typer: data

▶ `data Ret = Op | Ne | Ve | Ho deriving (Eq, Ord, Read, Show)`

▶ `type Pos = (Int, Int)`

▶ `mv :: Pos -> Ret -> Pos`

`mv (x,y) Ve = (x-1,y)   mv (x,y) Ho = (x+1,y)`

`mv (x,y) Op = (x,y+1)   mv (x,y) Ne = (x,y-1)`

▶ `mvs :: Pos -> [Ret] -> Pos`

- - `mvs p [a,b,c...] = fra p: først a, deretter b (fra mv p a), så c`

`mvs p [] = p`

`mvs p (r:rs) = mvs (mv p r) rs`

`... = (mv (mv (mv p a) b) c)... = ..... foldl mv p [a,b,c...]`

`!?! foldl mv p ms = mvs p ms`

Egendefinerte nye typer: data

► data Ret = Op | Ne | Ve | Ho deriving (Eq, Ord, Read, Show)

► type Pos = (Int, Int)

► mv :: Pos -> Ret -> Pos

mv (x,y) Ve = (x-1,y) mv (x,y) Ho = (x+1,y)

mv (x,y) Op = (x,y+1) mv (x,y) Ne = (x,y-1)

► mvs :: Pos -> [Ret] -> Pos

- - mvs p [a,b,c...] = fra p: først a, deretter b (fra mv p a), så c

mvs p [] = p

mvs p (r:rs) = mvs (mv p r) rs

... = (mv (mv (mv p a) b) c)... = foldl mv p [a,b,c...]

!?! foldl mv p ms = mvs p ms

foldl mv p [] = p =

Egendefinerte nye typer: data

▶ data Ret = Op | Ne | Ve | Ho deriving (Eq, Ord, Read, Show)

▶ type Pos = (Int, Int)

▶ mv :: Pos -> Ret -> Pos

mv (x,y) Ve = (x-1,y) mv (x,y) Ho = (x+1,y)

mv (x,y) Op = (x,y+1) mv (x,y) Ne = (x,y-1)

▶ mvs :: Pos -> [Ret] -> Pos

- - mvs p [a,b,c...] = fra p: først a, deretter b (fra mv p a), så c

mvs p [] = p

mvs p (r:rs) = mvs (mv p r) rs

... = (mv (mv (mv p a) b) c)... = foldl mv p [a,b,c...]

!?! foldl mv p ms = mvs p ms

foldl mv p [] = p = mvs p []

Egendefinerte nye typer: data

► data Ret = Op | Ne | Ve | Ho deriving (Eq, Ord, Read, Show)

► type Pos = (Int, Int)

► mv :: Pos -> Ret -> Pos

mv (x,y) Ve = (x-1,y) mv (x,y) Ho = (x+1,y)

mv (x,y) Op = (x,y+1) mv (x,y) Ne = (x,y-1)

► mvs :: Pos -> [Ret] -> Pos

- - mvs p [a,b,c...] = fra p: først a, deretter b (fra mv p a), så c

mvs p [] = p

mvs p (r:rs) = mvs (mv p r) rs

... = (mv (mv (mv p a) b) c)... = foldl mv p [a,b,c...]

!?! foldl mv p ms = mvs p ms

foldl mv p [] = p = mvs p []

foldl mv p (r:rs) = foldl mv (mv p r) rs =

Egendefinerte nye typer: data

▶ data Ret = Op | Ne | Ve | Ho deriving (Eq, Ord, Read, Show)

▶ type Pos = (Int, Int)

▶ mv :: Pos -> Ret -> Pos

mv (x,y) Ve = (x-1,y) mv (x,y) Ho = (x+1,y)

mv (x,y) Op = (x,y+1) mv (x,y) Ne = (x,y-1)

▶ mvs :: Pos -> [Ret] -> Pos

- - mvs p [a,b,c...] = fra p: først a, deretter b (fra mv p a), så c

mvs p [] = p

mvs p (r:rs) = mvs (mv p r) rs

... = (mv (mv (mv p a) b) c)... = foldl mv p [a,b,c...]

!?! foldl mv p ms = mvs p ms

foldl mv p [] = p = mvs p []

foldl mv p (r:rs) = foldl mv (mv p r) rs = mvs (mv p r) rs

Egendefinerte nye typer: data

▶ data Ret = Op | Ne | Ve | Ho deriving (Eq, Ord, Read, Show)

▶ type Pos = (Int, Int)

▶ mv :: Pos -> Ret -> Pos

mv (x,y) Ve = (x-1,y) mv (x,y) Ho = (x+1,y)

mv (x,y) Op = (x,y+1) mv (x,y) Ne = (x,y-1)

▶ mvs :: Pos -> [Ret] -> Pos

- - mvs p [a,b,c...] = fra p: først a, deretter b (fra mv p a), så c

mvs p [] = p

mvs p (r:rs) = mvs (mv p r) rs

... = (mv (mv (mv p a) b) c)... = foldl mv p [a,b,c...]

!?! foldl mv p ms = mvs p ms

foldl mv p [] = p = mvs p []

foldl mv p (r:rs) = foldl mv (mv p r) rs = IH mvs (mv p r) rs

Egendefinerte nye typer: data

▶ data Ret = Op | Ne | Ve | Ho deriving (Eq, Ord, Read, Show)

▶ type Pos = (Int, Int)

▶ mv :: Pos -> Ret -> Pos

mv (x,y) Ve = (x-1,y) mv (x,y) Ho = (x+1,y)

mv (x,y) Op = (x,y+1) mv (x,y) Ne = (x,y-1)

▶ mvs :: Pos -> [Ret] -> Pos

- - mvs p [a,b,c...] = fra p: først a, deretter b (fra mv p a), så c

mvs p [] = p

mvs p (r:rs) = mvs (mv p r) rs

... = (mv (mv (mv p a) b) c)... = foldl mv p [a,b,c...]

!?! foldl mv p ms = mvs p ms

foldl mv p [] = p = mvs p []

foldl mv p (r:rs) = foldl mv (mv p r) rs =IH mvs (mv p r) rs

= mvs p (r:rs)

Egendefinerte nye typer: data

▶ data Ret = Op | Ne | Ve | Ho deriving (Eq, Ord, Read, Show)

▶ type Pos = (Int, Int)

▶ mv :: Pos -> Ret -> Pos

mv (x,y) Ve = (x-1,y) mv (x,y) Ho = (x+1,y)

mv (x,y) Op = (x,y+1) mv (x,y) Ne = (x,y-1)

▶ mvs :: Pos -> [Ret] -> Pos

- - mvs p [a,b,c...] = fra p: først a, deretter b (fra mv p a), så c

mvs p [] = p

mvs p (r:rs) = mvs (mv p r) rs

... = (mv (mv (mv p a) b) c)... = foldl mv p [a,b,c...]

!?! foldl mv p ms = mvs p ms

foldl mv p [] = p = mvs p []

foldl mv p (r:rs) = foldl mv (mv p r) rs =IH mvs (mv p r) rs

= mvs p (r:rs)

Egendefinerte nye typer: data

▶ data Ret = Op | Ne | Ve | Ho deriving (Eq, Ord, Read, Show)

▶ type Pos = (Int, Int)

▶ mv :: Pos -> Ret -> Pos

mv (x,y) Ve = (x-1,y) mv (x,y) Ho = (x+1,y)

mv (x,y) Op = (x,y+1) mv (x,y) Ne = (x,y-1)

▶ mvs :: Pos -> [Ret] -> Pos

- - mvs p [a,b,c...] = fra p: først a, deretter b (fra mv p a), så c

mvs p [] = p

mvs p (r:rs) = mvs (mv p r) rs

... = (mv (mv (mv p a) b) c)... = foldl mv p [a,b,c...]

!?! foldl mv p ms = mvs p ms

foldl mv p [] = p = mvs p []

foldl mv p (r:rs) = foldl mv (mv p r) rs =IH mvs (mv p r) rs
= mvs p (r:rs)

Egendefinerte nye typer: data

▶ data Ret = Op | Ne | Ve | Ho deriving (Eq, Ord, Read, Show)

▶ type Pos = (Int, Int)

▶ mv :: Pos -> Ret -> Pos

mv (x,y) Ve = (x-1,y) mv (x,y) Ho = (x+1,y)

mv (x,y) Op = (x,y+1) mv (x,y) Ne = (x,y-1)

▶ mvs :: Pos -> [Ret] -> Pos

- - mvs p [a,b,c...] = fra p: først a, deretter b (fra mv p a), så c

mvs p [] = p

mvs p (r:rs) = mvs (mv p r) rs

... = (mv (mv (mv p a) b) c)... = foldl mv p [a,b,c...]

!?! foldl mv p ms = mvs p ms

foldl mv p [] = p = mvs p []

foldl mv p (r:rs) = foldl mv (mv p r) rs =IH mvs (mv p r) rs
= foldr mv p (r:rs) = mvs p (r:rs)

Egendefinerte nye typer: data

▶ data Ret = Op | Ne | Ve | Ho deriving (Eq, Ord, Read, Show)

▶ type Pos = (Int, Int)

▶ mv :: Pos -> Ret -> Pos

mv (x,y) Ve = (x-1,y) mv (x,y) Ho = (x+1,y)

mv (x,y) Op = (x,y+1) mv (x,y) Ne = (x,y-1)

▶ mvs :: Pos -> [Ret] -> Pos

- - mvs p [a,b,c...] = fra p: først a, deretter b (fra mv p a), så c

mvs p [] = p

mvs p (r:rs) = mvs (mv p r) rs

... = (mv (mv (mv p a) b) c)... = foldl mv p [a,b,c...]

!?! foldl mv p ms = mvs p ms

foldl mv p [] = p = mvs p []

foldl mv p (r:rs) = foldl mv (mv p r) rs =IH mvs (mv p r) rs
= foldr (flip mv) p (r:rs) = mvs p (r:rs)

Egendefinerte nye typer: data

▶ data Ret = Op | Ne | Ve | Ho deriving (Eq, Ord, Read, Show)

▶ type Pos = (Int, Int)

▶ mv :: Pos -> Ret -> Pos

mv (x,y) Ve = (x-1,y) mv (x,y) Ho = (x+1,y)

mv (x,y) Op = (x,y+1) mv (x,y) Ne = (x,y-1)

▶ mvs :: Pos -> [Ret] -> Pos

- - mvs p [a,b,c...] = fra p: først a, deretter b (fra mv p a), så c

mvs p [] = p

mvs p (r:rs) = mvs (mv p r) rs

... = (mv (mv (mv p a) b) c)... = foldl mv p [a,b,c...]

!?! foldl mv p ms = mvs p ms

foldl mv p [] = p = mvs p []

foldl mv p (r:rs) = foldl mv (mv p r) rs =IH mvs (mv p r) rs

(ulike veier) = foldr (flip mv) p (r:rs) = mvs p (r:rs)

Egendefinierte rekursive datatyper

► data Tre = Leaf Int | Node Tre Int Tre

Egendefinierte **rekursive** datatyper

► `data Tre = Leaf Int | Node Tre Int Tre`

`list (Leaf y) = [y]`

`list (Node l y r) = list l ++ [y] ++ list r`

Egendefinierte **rekursive** datatyper

► `data Tre = Leaf Int | Node Tre Int Tre`

`list (Leaf y) = [y]`

`list (Node l y r) = list l ++ [y] ++ list r`

`insert x (Leaf y) = Node ? y ?`

Egendefinierte **rekursive** datatyper

► `data Tre = Leaf Int | Node Tre Int Tre | Emp`

`list (Leaf y) = [y]`

`list (Node l y r) = list l ++ [y] ++ list r`

`insert x (Leaf y) = Node ? y ?`

Egendefinierte **rekursive** datatyper

► `data Tre = Leaf Int | Node Tre Int Tre | Emp`

`list (Leaf y) = [y]`

`list (Node l y r) = list l ++ [y] ++ list r`

`insert x (Leaf y) = Node (Leaf x) y Emp`

Egendefinierte rekursive datatyper

► `data Tre = Leaf Int | Node Tre Int Tre | Emp`

`list (Leaf y) = [y]`

`list (Node l y r) = list l ++ [y] ++ list r`

► `insert x Emp = Leaf x`

`insert x (Leaf y) = Node (Leaf x) y Emp`

Egendefinierte rekursive datatyper

- ▶ `data Tre = Leaf Int | Node Tre Int Tre | Emp`
- ▶ `list Emp = ??`
`list (Leaf y) = [y]`
`list (Node l y r) = list l ++ [y] ++ list r`
- ▶ `insert x Emp = Leaf x`
`insert x (Leaf y) = Node (Leaf x) y Emp`

Egendefinierte rekursive datatyper

- ▶ `data Tre = Leaf Int | Node Tre Int Tre | Emp`
- ▶ `list Emp = []`
`list (Leaf y) = [y]`
`list (Node l y r) = list l ++ [y] ++ list r`
- ▶ `insert x Emp = Leaf x`
`insert x (Leaf y) = Node (Leaf x) y Emp`

Egendefinierte **rekursive** datatyper

- ▶ `data Tre = Leaf Int | Node Tre Int Tre | Emp`
- ▶ `list Emp = []`
`list (Leaf y) = [y]`
`list (Node l y r) = list l ++ [y] ++ list r`
- ▶ `insert x Emp = Leaf x`
`insert x (Leaf y) = Node (Leaf x) y Emp`
`insert x (Node v i h) =`

Egendefinierte **rekursive** datatyper

- ▶ `data Tre = Leaf Int | Node Tre Int Tre | Emp`
- ▶ `list Emp = []`
`list (Leaf y) = [y]`
`list (Node l y r) = list l ++ [y] ++ list r`
- ▶ `insert x Emp = Leaf x`
`insert x (Leaf y) = Node (Leaf x) y Emp`
`insert x (Node v i h) = let ny = insert x v in (Node ny i h)`

Egendefinierte **rekursive** datatyper

- ▶ `data Tre = Leaf Int | Node Tre Int Tre | Emp`
- ▶ `list Emp = []`
`list (Leaf y) = [y]`
`list (Node l y r) = list l ++ [y] ++ list r`
- ▶ `insert x Emp = Leaf x`
`insert x (Leaf y) = Node (Leaf x) y Emp`
`insert x (Node v i h) = let ny = insert x v in (Node ny i h)`
- ▶ `sum Emp = 0`
`sum (Leaf x) = x`
`sum (Node v i h) = sum v + i + sum h`

Egendefinierte **rekursive** datatyper

- ▶ `data Tre = Leaf Int | Node Tre Int Tre | Emp`
- ▶ `list Emp = []`
`list (Leaf y) = [y]`
`list (Node l y r) = list l ++ [y] ++ list r`
- ▶ `insert x Emp = Leaf x`
`insert x (Leaf y) = Node (Leaf x) y Emp`
`insert x (Node v i h) = let ny = insert x v in (Node ny i h)`
- ▶ `sum Emp = 0`
`sum (Leaf x) = x`
`sum (Node v i h) = sum v + i + sum h`

`sum = foldt`

Egendefinerte **rekursive** datatyper

- ▶ `data Tre = Leaf Int | Node Tre Int Tre | Emp`
 - ▶ `list Emp = []`
`list (Leaf y) = [y]`
`list (Node l y r) = list l ++ [y] ++ list r`
 - ▶ `insert x Emp = Leaf x`
`insert x (Leaf y) = Node (Leaf x) y Emp`
`insert x (Node v i h) = let ny = insert x v in (Node ny i h)`
 - ▶ `sum Emp = 0`
`sum (Leaf x) = x`
`sum (Node v i h) = sum v + i + sum h`
- `sum = foldt 0`

Egendefinerte **rekursive** datatyper

- ▶ `data Tre = Leaf Int | Node Tre Int Tre | Emp`
- ▶ `list Emp = []`
`list (Leaf y) = [y]`
`list (Node l y r) = list l ++ [y] ++ list r`
- ▶ `insert x Emp = Leaf x`
`insert x (Leaf y) = Node (Leaf x) y Emp`
`insert x (Node v i h) = let ny = insert x v in (Node ny i h)`
- ▶ `sum Emp = 0`
`sum (Leaf x) = x`
`sum (Node v i h) = sum v + i + sum h`

`sum = foldt 0`

`list = foldt`

Egendefinerte **rekursive** datatyper

- ▶ `data Tre = Leaf Int | Node Tre Int Tre | Emp`
- ▶ `list Emp = []`
`list (Leaf y) = [y]`
`list (Node l y r) = list l ++ [y] ++ list r`
- ▶ `insert x Emp = Leaf x`
`insert x (Leaf y) = Node (Leaf x) y Emp`
`insert x (Node v i h) = let ny = insert x v in (Node ny i h)`
- ▶ `sum Emp = 0`
`sum (Leaf x) = x`
`sum (Node v i h) = sum v + i + sum h`

`sum = foldt 0`

`list = foldt []`

Egendefinerte **rekursive** datatyper

- ▶ `data Tre = Leaf Int | Node Tre Int Tre | Emp`
- ▶ `list Emp = []`
`list (Leaf y) = [y]`
`list (Node l y r) = list l ++ [y] ++ list r`
- ▶ `insert x Emp = Leaf x`
`insert x (Leaf y) = Node (Leaf x) y Emp`
`insert x (Node v i h) = let ny = insert x v in (Node ny i h)`
- ▶ `sum Emp = 0`
`sum (Leaf x) = x`
`sum (Node v i h) = sum v + i + sum h`

`sum = foldt 0 id`

`list = foldt []`

Egendefinerte **rekursive** datatyper

- ▶ `data Tre = Leaf Int | Node Tre Int Tre | Emp`
- ▶ `list Emp = []`
`list (Leaf y) = [y]`
`list (Node l y r) = list l ++ [y] ++ list r`
- ▶ `insert x Emp = Leaf x`
`insert x (Leaf y) = Node (Leaf x) y Emp`
`insert x (Node v i h) = let ny = insert x v in (Node ny i h)`
- ▶ `sum Emp = 0`
`sum (Leaf x) = x`
`sum (Node v i h) = sum v + i + sum h`

`sum = foldt 0 id`

`list = foldt [] (\ y -> [y])`

Egendefinerte **rekursive** datatyper

- ▶ `data Tre = Leaf Int | Node Tre Int Tre | Emp`
- ▶ `list Emp = []`
`list (Leaf y) = [y]`
`list (Node l y r) = list l ++ [y] ++ list r`
- ▶ `insert x Emp = Leaf x`
`insert x (Leaf y) = Node (Leaf x) y Emp`
`insert x (Node v i h) = let ny = insert x v in (Node ny i h)`
- ▶ `sum Emp = 0`
`sum (Leaf x) = x`
`sum (Node v i h) = sum v + i + sum h`

`sum = foldt 0 id`

`list = foldt [] (:[])`

Egendefinierte **rekursive** datatyper

- ▶ `data Tre = Leaf Int | Node Tre Int Tre | Emp`
 - ▶ `list Emp = []`
`list (Leaf y) = [y]`
`list (Node l y r) = list l ++ [y] ++ list r`
 - ▶ `insert x Emp = Leaf x`
`insert x (Leaf y) = Node (Leaf x) y Emp`
`insert x (Node v i h) = let ny = insert x v in (Node ny i h)`
 - ▶ `sum Emp = 0`
`sum (Leaf x) = x`
`sum (Node v i h) = sum v + i + sum h`
- `sum = foldt 0 id (\ vr i hr -> vr + i + hr)`
- `list = foldt [] (:[])`

Egendefinierte **rekursive** datatyper

- ▶ `data Tre = Leaf Int | Node Tre Int Tre | Emp`
 - ▶ `list Emp = []`
`list (Leaf y) = [y]`
`list (Node l y r) = list l ++ [y] ++ list r`
 - ▶ `insert x Emp = Leaf x`
`insert x (Leaf y) = Node (Leaf x) y Emp`
`insert x (Node v i h) = let ny = insert x v in (Node ny i h)`
 - ▶ `sum Emp = 0`
`sum (Leaf x) = x`
`sum (Node v i h) = sum v + i + sum h`
- `sum = foldt 0 id (\ vr i hr -> vr + i + hr)`
- `list = foldt [] (:[]) (\ vr y hr -> vr ++ [y] ++ hr)`

Egendefinierte rekursive datatyper

- ▶ `data Tre = Leaf Int | Node Tre Int Tre | Emp`
 - ▶ `list Emp = []`
`list (Leaf y) = [y]`
`list (Node l y r) = list l ++ [y] ++ list r`
 - ▶ `insert x Emp = Leaf x`
`insert x (Leaf y) = Node (Leaf x) y Emp`
`insert x (Node v i h) = let ny = insert x v in (Node ny i h)`
 - ▶ `sum Emp = 0`
`sum (Leaf x) = x`
`sum (Node v i h) = sum v + i + sum h`
- `sum = foldt 0 id (\ vr i hr -> vr + i + hr)`
- `list = foldt [] (:[]) (\ vr y hr -> vr ++ [y] ++ hr)`
- ▶ `foldt tm bl nd Emp =`
`foldt tm bl nd (Leaf x) =`
`foldt t b n (Node v i h) =`

Egendefinierte **rekursive** datatyper

- ▶ `data Tre = Leaf Int | Node Tre Int Tre | Emp`
 - ▶ `list Emp = []`
`list (Leaf y) = [y]`
`list (Node l y r) = list l ++ [y] ++ list r`
 - ▶ `insert x Emp = Leaf x`
`insert x (Leaf y) = Node (Leaf x) y Emp`
`insert x (Node v i h) = let ny = insert x v in (Node ny i h)`
 - ▶ `sum Emp = 0`
`sum (Leaf x) = x`
`sum (Node v i h) = sum v + i + sum h`
- `sum = foldt 0 id (\ vr i hr -> vr + i + hr)`
- `list = foldt [] (:[]) (\ vr y hr -> vr ++ [y] ++ hr)`
- ▶ `foldt tm bl nd Emp = tm`
`foldt tm bl nd (Leaf x) =`
`foldt t b n (Node v i h) =`

Egendefinerte **rekursive** datatyper

- ▶ `data Tre = Leaf Int | Node Tre Int Tre | Emp`
 - ▶ `list Emp = []`
`list (Leaf y) = [y]`
`list (Node l y r) = list l ++ [y] ++ list r`
 - ▶ `insert x Emp = Leaf x`
`insert x (Leaf y) = Node (Leaf x) y Emp`
`insert x (Node v i h) = let ny = insert x v in (Node ny i h)`
 - ▶ `sum Emp = 0`
`sum (Leaf x) = x`
`sum (Node v i h) = sum v + i + sum h`
- `sum = foldt 0 id (\ vr i hr -> vr + i + hr)`
- `list = foldt [] (:[]) (\ vr y hr -> vr ++ [y] ++ hr)`
- ▶ `foldt tm bl nd Emp = tm`
`foldt tm bl nd (Leaf x) = bl x`
`foldt t b n (Node v i h) =`

Egendefinierte **rekursive** datatyper

- ▶ `data Tre = Leaf Int | Node Tre Int Tre | Emp`
 - ▶ `list Emp = []`
`list (Leaf y) = [y]`
`list (Node l y r) = list l ++ [y] ++ list r`
 - ▶ `insert x Emp = Leaf x`
`insert x (Leaf y) = Node (Leaf x) y Emp`
`insert x (Node v i h) = let ny = insert x v in (Node ny i h)`
 - ▶ `sum Emp = 0`
`sum (Leaf x) = x`
`sum (Node v i h) = sum v + i + sum h`
- `sum = foldt 0 id (\ vr i hr -> vr + i + hr)`
- `list = foldt [] (:[]) (\ vr y hr -> vr ++ [y] ++ hr)`
- ▶ `foldt tm bl nd Emp = tm`
`foldt tm bl nd (Leaf x) = bl x`
`foldt t b n (Node v i h) = n (foldt t b n v) i (foldt t b n h)`

Egendefinierte **rekursive** datatyper

- ▶ `data Tre = Leaf Int | Node Tre Int Tre | Emp`
 - ▶ `list Emp = []`
`list (Leaf y) = [y]`
`list (Node l y r) = list l ++ [y] ++ list r`
 - ▶ `insert x Emp = Leaf x`
`insert x (Leaf y) = Node (Leaf x) y Emp`
`insert x (Node v i h) = let ny = insert x v in (Node ny i h)`
 - ▶ `sum Emp = 0`
`sum (Leaf x) = x`
`sum (Node v i h) = sum v + i + sum h`
- `sum = foldt 0 id (\ vr i hr -> vr + i + hr)`
- `list = foldt [] (:[]) (\ vr y hr -> vr ++ [y] ++ hr)`
- ▶ `foldt tm bl nd Emp = tm`
`foldt tm bl nd (Leaf x) = bl x`
`foldt t b n (Node v i h) = n (foldt t b n v) i (foldt t b n h)`

`fins a = foldt`

Egendefinierte **rekursive** datatyper

- ▶ `data Tre = Leaf Int | Node Tre Int Tre | Emp`
- ▶ `list Emp = []`
`list (Leaf y) = [y]`
`list (Node l y r) = list l ++ [y] ++ list r`
- ▶ `insert x Emp = Leaf x`
`insert x (Leaf y) = Node (Leaf x) y Emp`
`insert x (Node v i h) = let ny = insert x v in (Node ny i h)`
- ▶ `sum Emp = 0`
`sum (Leaf x) = x`
`sum (Node v i h) = sum v + i + sum h`

`sum = foldt 0 id (\ vr i hr -> vr + i + hr)`

`list = foldt [] (:[]) (\ vr y hr -> vr++[y]++hr)`

- ▶ `foldt tm bl nd Emp = tm`
`foldt tm bl nd (Leaf x) = bl x`
`foldt t b n (Node v i h) = n (foldt t b n v) i (foldt t b n h)`

`fins a = foldt False`

Egendefinierte **rekursive** datatyper

- ▶ `data Tre = Leaf Int | Node Tre Int Tre | Emp`
- ▶ `list Emp = []`
`list (Leaf y) = [y]`
`list (Node l y r) = list l ++ [y] ++ list r`
- ▶ `insert x Emp = Leaf x`
`insert x (Leaf y) = Node (Leaf x) y Emp`
`insert x (Node v i h) = let ny = insert x v in (Node ny i h)`
- ▶ `sum Emp = 0`
`sum (Leaf x) = x`
`sum (Node v i h) = sum v + i + sum h`

`sum = foldt 0 id (\ vr i hr -> vr + i + hr)`

`list = foldt [] (:[]) (\ vr y hr -> vr++[y]++hr)`

- ▶ `foldt tm bl nd Emp = tm`
`foldt tm bl nd (Leaf x) = bl x`
`foldt t b n (Node v i h) = n (foldt t b n v) i (foldt t b n h)`

`fins a = foldt False (==a)`

Egendefinierte rekursive datatyper

- ▶ `data Tre = Leaf Int | Node Tre Int Tre | Emp`
- ▶ `list Emp = []`
`list (Leaf y) = [y]`
`list (Node l y r) = list l ++ [y] ++ list r`
- ▶ `insert x Emp = Leaf x`
`insert x (Leaf y) = Node (Leaf x) y Emp`
`insert x (Node v i h) = let ny = insert x v in (Node ny i h)`
- ▶ `sum Emp = 0`
`sum (Leaf x) = x`
`sum (Node v i h) = sum v + i + sum h`

`sum = foldt 0 id (\ vr i hr -> vr + i + hr)`

`list = foldt [] (:[]) (\ vr y hr -> vr++[y]++hr)`

- ▶ `foldt tm bl nd Emp = tm`
`foldt tm bl nd (Leaf x) = bl x`
`foldt t b n (Node v i h) = n (foldt t b n v) i (foldt t b n h)`

`fins a = foldt False (==a) (\ v i h -> (i==n) || v || h)`

Egendefinerte parametrisererte typer

```
data Tre = Emp | Leaf Int | Node Tre Int Tre
```

Egendefinerte parametriserte typer

```
data Tre = Emp | Leaf Int | Node Tre Int Tre
```

```
data Tre t = Leaf t | Node (Tre t) (Tre t)
```

Egendefinerte parametriserte typer

```
data Tre = Emp | Leaf Int | Node Tre Int Tre
```

```
data Tre t = Leaf t | Node (Tre t) (Tre t)
```

Egendefinerte parametriserte typer

```
data Tre = Emp | Leaf Int | Node Tre Int Tre
```

```
data Tre t = Leaf t | Node (Tre t) (Tre t) .. data kun ved blader
```

Egendefinerte **parametriserte** typer

```
data Tre = Emp | Leaf Int | Node Tre Int Tre
```

```
data Tre t = Leaf t | Node (Tre t) (Tre t) .. data kun ved blader  
= Leaf | Node (Tre t) t (Tre t)
```

Egendefinerte **parametriserte** typer

```
data Tre = Emp | Leaf Int | Node Tre Int Tre
```

```
data Tre t = Leaf t | Node (Tre t) (Tre t) .. data kun ved blader  
= Leaf | Node (Tre t) t (Tre t)
```

Egendefinerte **parametriserte** typer

```
data Tre = Emp | Leaf Int | Node Tre Int Tre
```

```
data Tre t = Leaf t | Node (Tre t) (Tre t) .. data kun ved blader  
              = Leaf | Node (Tre t) t (Tre t) .... kun ved interne noder
```

Egendefinerte **parametriserte** typer

```
data Tre = Emp | Leaf Int | Node Tre Int Tre
```

```
data Tre t = Leaf t | Node (Tre t) (Tre t) .. data kun ved blader  
           = Leaf | Node (Tre t) t (Tre t) .... kun ved interne noder  
           = Node t [ Tre t ]
```

Egendefinerte parametriserte typer

```
data Tre = Emp | Leaf Int | Node Tre Int Tre
```

```
data Tre t = Leaf t | Node (Tre t) (Tre t) .. data kun ved blader  
           = Leaf | Node (Tre t) t (Tre t) .... kun ved interne noder  
           = Node t [ Tre t ]
```

Egendefinerte **parametriserte** typer

```
data Tre = Emp | Leaf Int | Node Tre Int Tre
```

```
data Tre t = Leaf t | Node (Tre t) (Tre t) .. data kun ved blader  
            = Leaf | Node (Tre t) t (Tre t) .... kun ved interne noder  
            = Node t [ Tre t ] .. flere barn (et blad har tom barneliste)
```

Egendefinerte **parametriserte** typer

```
data Tre = Emp | Leaf Int | Node Tre Int Tre
```

```
data Tre t = Leaf t | Node (Tre t) (Tre t) .. data kun ved blader  
           = Leaf | Node (Tre t) t (Tre t) .... kun ved interne noder  
           = Node t [ Tre t ] .. flere barn (et blad har tom barneliste)
```

Egendefinerte **parametriserte** typer

```
data Tre = Emp | Leaf Int | Node Tre Int Tre
```

```
data Tre t = Leaf t | Node (Tre t) (Tre t) .. data kun ved blader  
           = Leaf | Node (Tre t) t (Tre t) .... kun ved interne noder  
           = Node t [ Tre t ] .. flere barn (et blad har tom barneliste)
```

► left :: Tre t -> Tre t
left (Leaf x) =

Egendefinerte **parametriserte** typer

```
data Tre = Emp | Leaf Int | Node Tre Int Tre
```

```
data Tre t = Leaf t | Node (Tre t) (Tre t) .. data kun ved blader  
           = Leaf | Node (Tre t) t (Tre t) .... kun ved interne noder  
           = Node t [ Tre t ] .. flere barn (et blad har tom barneliste)
```

► `left :: Tre t -> Tre t`
`left (Leaf x) = ???`

Egendefinerte **parametriserte** typer

```
data Tre = Emp | Leaf Int | Node Tre Int Tre
```

```
data Tre t = Leaf t | Node (Tre t) (Tre t) .. data kun ved blader  
           = Leaf | Node (Tre t) t (Tre t) .... kun ved interne noder  
           = Node t [ Tre t ] .. flere barn (et blad har tom barneliste)
```

► left :: Tre t -> Tre t
left (Leaf x) = **Emp**

Egendefinerte **parametriserte** typer

```
data Tre = Emp | Leaf Int | Node Tre Int Tre
```

```
data Tre t = Leaf t | Node (Tre t) (Tre t) .. data kun ved blader  
           = Leaf | Node (Tre t) t (Tre t) .... kun ved interne noder  
           = Node t [ Tre t ] .. flere barn (et blad har tom barneliste)
```

► left :: Tre t -> Tre t
left (Leaf x) = **Nothing**

Egendefinerte **parametriserte** typer

```
data Tre = Emp | Leaf Int | Node Tre Int Tre
```

```
data Tre t = Leaf t | Node (Tre t) (Tre t) .. data kun ved blader  
           = Leaf | Node (Tre t) t (Tre t) .... kun ved interne noder  
           = Node t [ Tre t ] .. flere barn (et blad har tom barneliste)
```

```
▶ left :: Tre t -> Tre t  
left (Leaf x) = Nothing  
left (Leaf Emp) = Nothing  
left (Node ve x ho) = Just ve
```

Egendefinerte **parametriserte** typer

```
data Tre = Emp | Leaf Int | Node Tre Int Tre
```

```
data Tre t = Leaf t | Node (Tre t) (Tre t) .. data kun ved blader  
           = Leaf | Node (Tre t) t (Tre t) .... kun ved interne noder  
           = Node t [ Tre t ] .. flere barn (et blad har tom barneliste)
```

- ▶ `left :: Tre t -> Tre t`
`left (Leaf x) = Nothing`
`left (Leaf Emp) = Nothing`
`left (Node ve x ho) = Just ve`

- ▶ `data Maybe t = Nothing | Just t`
for partielle funksjoner – som returnerer verdi av type `t`

Egendefinerte **parametriserte** typer

```
data Tre = Emp | Leaf Int | Node Tre Int Tre
```

```
data Tre t = Leaf t | Node (Tre t) (Tre t) .. data kun ved blader  
           = Leaf | Node (Tre t) t (Tre t) .... kun ved interne noder  
           = Node t [ Tre t ] .. flere barn (et blad har tom barneliste)
```

- ▶ `left :: Tre t -> Maybe (Tre t)` (import Data.Maybe)
`left (Leaf x) = Nothing`
`left (Leaf Emp) = Nothing`
`left (Node ve x ho) = Just ve`
- ▶ `data Maybe t = Nothing | Just t`
for partielle funksjoner – som returnerer verdi av type t

Enklest feilhåndtering

- ▶ `import Data.Maybe`
`data Maybe t = Nothing | Just t`
– for partielle funksjoner som returnerer verdi av type `t`

Enklest feilhåndtering

- ▶ `import Data.Maybe`
`data Maybe t = Nothing | Just t`
– for partielle funksjoner som returnerer verdi av type `t`

```
phead [] = Nothing
```

```
phead (x:xs) = x
```

Enklest feilhåndtering

- ▶ `import Data.Maybe`
`data Maybe t = Nothing | Just t`
– for partielle funksjoner som returnerer verdi av type `t`

`phead [] = Nothing`
`phead (x:xs) = Just x`

Enklest feilhåndtering

- ▶ import Data.Maybe
data Maybe **t** = Nothing | Just **t**
– for partielle funksjoner som returnerer verdi av type t

```
phead [] = Nothing  
phead (x:xs) = Just x
```

```
phead :: [t] -> Maybe t  
når x :: t
```

Enklest feilhåndtering

- ▶ `import Data.Maybe`
`data Maybe t = Nothing | Just t`
– for partielle funksjoner som returnerer verdi av type `t`

<code>phead [] = Nothing</code>	<code>phead :: [t] -> Maybe t</code>
<code>phead (x:xs) = Just x</code>	når <code>x :: t</code>
- ▶ krever tilbakekast for å få riktig type av et riktig resultat, f.eks.

<code>re (Just x) = x,</code>	<code>re :: Maybe t -> t</code>
-------------------------------	------------------------------------

som må ofte brukes der `Maybe`-verdien kan bli produsert

Enklest feilhåndtering

- ▶ `import Data.Maybe`
`data Maybe t = Nothing | Just t`
– for partielle funksjoner som returnerer verdi av type `t`

<code>phead [] = Nothing</code>	<code>phead :: [t] -> Maybe t</code>
<code>phead (x:xs) = Just x</code>	når <code>x :: t</code>
- ▶ krever tilbakekast for å få riktig type av et riktig resultat, f.eks.
`re (Just x) = x,` `re :: Maybe t -> t`
som må ofte brukes der Maybe-verdien kan bli produsert
- ▶ `(read "12")` `read :: Read t => String -> t`

Enklest feilhåndtering

- ▶ `import Data.Maybe`
`data Maybe t = Nothing | Just t`
– for partielle funksjoner som returnerer verdi av type `t`

<code>phead [] = Nothing</code>	<code>phead :: [t] -> Maybe t</code>
<code>phead (x:xs) = Just x</code>	når <code>x :: t</code>

- ▶ krever tilbakekast for å få riktig type av et riktig resultat, f.eks.
`re (Just x) = x,` `re :: Maybe t -> t`
som må ofte brukes der Maybe-verdien kan bli produsert
- ▶ `(read "12")` `read :: Read t => String -> t`
*** Exception: Prelude.read: no parse

Enklest feilhåndtering

- ▶ `import Data.Maybe`
`data Maybe t = Nothing | Just t`
– for partielle funksjoner som returnerer verdi av type `t`

<code>phead [] = Nothing</code>	<code>phead :: [t] -> Maybe t</code>
<code>phead (x:xs) = Just x</code>	når <code>x :: t</code>
- ▶ krever tilbakekast for å få riktig type av et riktig resultat, f.eks.
`re (Just x) = x,` `re :: Maybe t -> t`
som må ofte brukes der Maybe-verdien kan bli produsert
- ▶ `(read "12") :: Int` `read :: Read t => String -> t`

Enklest feilhåndtering

- ▶ `import Data.Maybe`
`data Maybe t = Nothing | Just t`
– for partielle funksjoner som returnerer verdi av type `t`

<code>phead [] = Nothing</code>	<code>phead :: [t] -> Maybe t</code>
<code>phead (x:xs) = Just x</code>	når <code>x :: t</code>

- ▶ krever tilbakekast for å få riktig type av et riktig resultat, f.eks.
`re (Just x) = x,` `re :: Maybe t -> t`
som må ofte brukes der Maybe-verdien kan bli produsert
- ▶ `(read "12") :: Int` `read :: Read t => String -> t`
`12`

Enklest feilhåndtering

- ▶ `import Data.Maybe`
`data Maybe t = Nothing | Just t`
– for partielle funksjoner som returnerer verdi av type `t`

<code>phead [] = Nothing</code>	<code>phead :: [t] -> Maybe t</code>
<code>phead (x:xs) = Just x</code>	når <code>x :: t</code>

- ▶ krever tilbakekast for å få riktig type av et riktig resultat, f.eks.
`re (Just x) = x,` `re :: Maybe t -> t`
som må ofte brukes der Maybe-verdien kan bli produsert
- ▶ `(read "12") :: Int` `read :: Read t => String -> t`
`12`
- ▶ `phead [] = error "Tom liste"`

Enklest feilhåndtering

- ▶ `import Data.Maybe`
`data Maybe t = Nothing | Just t`
– for partielle funksjoner som returnerer verdi av type `t`

<code>phead [] = Nothing</code>	<code>phead :: [t] -> Maybe t</code>
<code>phead (x:xs) = Just x</code>	når <code>x :: t</code>

- ▶ krever tilbakekast for å få riktig type av et riktig resultat, f.eks.
`re (Just x) = x,` `re :: Maybe t -> t`
som må ofte brukes der Maybe-verdien kan bli produsert

- ▶ `(read "12") :: Int` `read :: Read t => String -> t`
`12`

- ▶ `phead [] = error "Tom liste"`

- ▶ `error :: String -> t`

Enklest feilhåndtering

- ▶ `import Data.Maybe`
`data Maybe t = Nothing | Just t`
– for partielle funksjoner som returnerer verdi av type `t`

<code>phead [] = Nothing</code>	<code>phead :: [t] -> Maybe t</code>
<code>phead (x:xs) = Just x</code>	når <code>x :: t</code>

- ▶ krever tilbakekast for å få riktig type av et riktig resultat, f.eks.
`re (Just x) = x,` `re :: Maybe t -> t`
som må ofte brukes der Maybe-verdien kan bli produsert

- ▶ `(read "12") :: Int` `read :: Read t => String -> t`
`12`

- ▶ `phead [] = error "Tom liste"`
`phead (x:xs) = x`
`phead :: [t] -> t`

- ▶ `error :: String -> t`